IEEE WIE ACT 5.0 · EdNova Technical Challenge

EdNova LLM Engine

Integration and technical guide

engine v1.3 (v1 frozen 2026-10-24; v1.1 2026-10-26; v1.2 and v1.3 2026-11-01)

A provider-agnostic engine that turns a learner's message into a classification, a hint, a grounded explanation, an exercise or a grade, while staying inside a slow, quota-limited, CPU-only LLM platform. This guide gives you the inputs and outputs you need to call it from your own project, and explains how every part works so you can customize or improve it.

At a glance

Interface	one async call: `await engine.run(task, inputs, context, priority)` → `Result`
Tasks	classify · hint · explain (RAG) · grade · quiz · compress (+ your own)
Model classes	*compact* (default for everything) and *mid-size* (explain only) (superseded in v1.3: three classes, with *vision* for `ocr_transcribe`; see section "What is new in v1.3)
Guarantee	never raises a provider error: every failure comes back as a degraded `Result` with a reason
Platform	any OpenAI-compatible API (FelCloud, Ollama); switching = editing `.env`
Tests	768 unit + 5 integration (the integration tests were not run for v1.2 and v1.3: Ollama stays off)
Measured	0 exceptions in 4 load profiles · 75–100 % of calls on compact · 28–61 % cache hits

Source: `/home/ibrahim/wie` · every number in this guide comes from the repository's tests, eval scripts and reports, as of the freeze.

Contents

1 What the engine is (and is not) 2

2 Quick start 2

2.1 Install and configure . . . 2

2.2 Your first calls . . . 3

3 Public interface 4

3.1 `Engine.create` . . . 4

3.2 `engine.run` . . . 4

3.3 `Result` . . . 5

3.4 Events . . . 6

4 Tasks: inputs and outputs 6

4.1 classify . . . 7

4.2 hint . . . 7

4.3 explain . . . 8

4.4 grade . . . 8

4.5 quiz . . . 9

4.6 compress . . . 9

5 Degrade reasons: the complete catalog 9

6 Inside `engine.run`: the request lifecycle 11

7 Gatekeeper: living inside the platform limits 12

7.1 Rate and concurrency are two mechanisms . . . 12

7.2 Priority queue . . . 12

7.3 The platform contract as code . . . 13

8 Router: compact first, mid-size by rule 13

9 Caching and deduplication 14

10 Retrieval (RAG) for explain 15

11 The deterministic layer 16

12 Metrics: what is measured 16

13 What is new in v1.3 17

13.1 Identity, fairness and the background reserve . . . 17
13.2 The vision class . . . 18
13.3 New tasks . . . 18
13.4 Course content . . . 19
13.5 Cache administration and scrubbing . . . 19
13.6 Verification status . . . 19

14 Measured behaviour (engine v1 local, v1.1 on FelCloud) 19
14.1 The same measurements on FelCloud (engine v1.1, 26 Oct) . . . 20

15 Configuration reference 22

16 Customizing the engine 24
16.1 Teach a new subject (no code) . . . 24
16.2 Change a prompt . . . 24
16.3 Add your own task . . . 25
16.4 Other common changes . . . 26

17 Known limitations and improvement ideas 26

A File map 27

1 What the engine is (and is not)

The engine is the **LLM/platform layer** of EdNova. It knows how to talk to a constrained LLM provider, how to choose between a small and a medium model, how to survive saturation and quota limits, how to ground an explanation in course material, and how to measure all of it. It does *not* know about learners, sessions, knowledge tracing or UI. Those belong to your **feature layer**, which calls the engine.

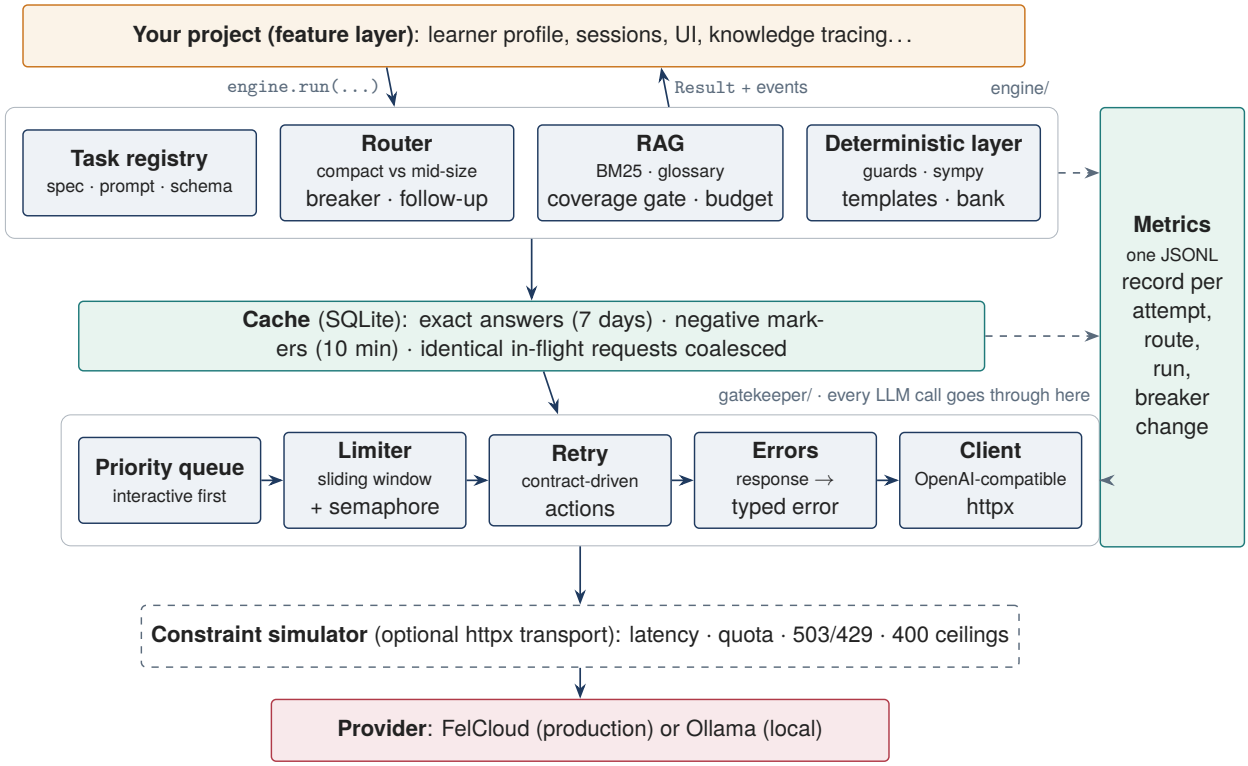


Figure 1: Layers of the engine. Requests flow down, results flow back up. Every component writes metrics.

Three properties drive the design, because they are what the challenge grades (orchestration 25 %, handling constraints 25 %, resilience 15 %):

1. **Fewer, cheaper requests.** Compact first; mid-size only for `explain`. Rules, templates, the exercise bank and `sympy` answer whatever they can with 0 requests. Identical requests are cached or coalesced.
2. **The platform contract is code.** Quota-exhausted, saturation, ceilings and quota headers each map to one typed error and one handling rule (§7).
3. **Degrade visibly, never fail.** `engine.run` does not raise provider errors. A result is always well-formed; when something went wrong it says so (`degraded=True` + a reason).

2 Quick start

2.1 Install and configure

Requirements: Python 3.10+ (the code avoids 3.11-only features), and the packages in `requirements.txt`: `httpx`, `pydantic` v2, `python-dotenv`, `pyyaml`, `sympy` (+ `pytest`, `pytest-asyncio` for tests).

```
pip install -r requirements.txt
cp .env.example .env            # then fill LLM_BASE_URL, LLM_API_KEY, MODEL_* (see
    section 13)
python -m scripts.index_docs    # chunk data/docs/*.md into data/index/chunks.json (no
     LLM call)
pytest -q                       # 768 unit tests, no LLM needed
python -m scripts.smoke         # one real call per task
python -m scripts.chat          # interactive console: one line = one learner turn
```

Four settings are required (`config.py` raises `ConfigError` without them): `LLM_BASE_URL`, `LLM_API_KEY`, `MODEL_COMPACT`, `MODEL_MIDSIZE`. Everything else has a default. `ENV_FILE` selects the profile file (default `.env`); variables already in the process environment override the file.

2.2 Your first calls

The engine is an **async Python library**, not a server. Create one `Engine` per process and reuse it: it owns the rate limiter, the queue, the circuit breaker and the cache connection.

```
import asyncio
from engine import Engine, Priority

async def main():
    async with Engine.create() as engine:          # reads ENV_FILE / .env
        c = await engine.run("classify", {"message": "chnowa ya3ni derivee? ma fhemt
    chay"},
                             {"turn_id": "demo-1"}, Priority.INTERACTIVE)
        follow = engine.router.follow_up(c.data)    # intent -> next task
        if follow.task == "explain":
            r = await engine.run("explain", {"question": "C'est quoi la dérivée d'un
    produit ?"},
                                 {"language": "FRENCH", "confusion": c.data["confusion
    "]})
            print(r.text)
            print(r.model_class_used, r.from_cache, r.degraded, r.degrade_reason)

asyncio.run(main())
```

Real output of these calls (local Ollama, simulator off, run while writing this guide):

```
classify -> {"intent": "EXPLAIN", "language": "ARABIZI", "confusion": "HIGH",
             "complexity": 2, "confidence": 0.7}       compact, 858 in / 39 out tokens
follow_up -> FollowUp(task='explain', reason='intent EXPLAIN -> explain', overridden=
    False)

explain.text ->
  Pas de souci, on reprend ce point calmement.                        <- opener (
    template)

  La dérivée d'un produit n'est pas le produit des dérivées. Pour     <- body (mid-
    size)
  trouver la dérivée d'un produit, on utilise la formule
  (u·v)' = u'·v + u·v'. [...] Par exemple, pour f(x) = x(x + 1),
  on trouve f'(x) = 2x + 1 en utilisant cette formule.

  À toi : Calculer la dérivée de f(x) = x(x + 3).                     <- bank exercise
    d05
explain -> midsize, from_cache=False, degraded=False, 35.5 s, 427 in / 138 out tokens
same question again (confusion LOW) -> from_cache=True, 1.2 ms, 0 tokens,
```

```
  opener rebuilt: "Voici l'essentiel du cours sur ce point."
```

Note how `classify` spotted Arabizi and raised confusion to `HIGH` (the learner wrote "ma fhemt chay"), and how the second `explain` came from the cache with a different opener: the cache stores only the model's body, the scaffolding is rebuilt for every learner (§4.3).

3 Public interface

3.1 `Engine.create`

```
Engine.create(settings=None, *, transport=None, client=None, sink=None, cache=None,
              registry=None, retriever=None, clock=time.monotonic, sleep=asyncio.sleep,
    rng=None)
```

Argument	Use it to…
`settings`	pass a `Settings` object instead of reading `.env` (`config.load_settings(env_dict)` builds one from a dict).
`transport`	inject an `httpx` transport (tests, a custom proxy). The simulator is added automatically when `SIMULATE_CONSTRAINTS=true`.
`client`	replace the whole HTTP client (e.g. a fake provider in tests).
`sink`	where metrics go: default `JsonlSink(METRICS_PATH)`; anything with `write(record)` works.
`cache`	an `ExactCache`; use `ExactCache(":memory:", ttl)` for throwaway runs.
`registry`	a task `Registry` with your own tasks added (§16.3).
`retriever`	a custom retriever object (§10).
`clock, sleep, rng`	virtual time and seeded randomness for deterministic tests.

Use it as `async with Engine.create() as engine:` or call `await engine.aclose()` at shutdown.

3.2 `engine.run`

```
result: Result = await engine.run(task, inputs=None, context=None,
                                  priority=Priority.INTERACTIVE, on_event=None)
```

Parameter	Meaning
`task`	`"classify" | "hint" | "explain" | "grade" | "quiz" | "compress"` or a task you registered. Unknown → `ValueError`.
`inputs`	task-specific dict (§4). Missing required input → `ValueError`, before any request.
`context`	learner/turn context (table below). Unknown keys are kept and passed to the task.
`priority`	`Priority.INTERACTIVE` (a learner is waiting) or `BACKGROUND` (prefetch, evals). Interactive requests always leave the queue first.
`on_event`	optional callback `f(Event)` called live as the call progresses: use it for staged progress in a UI.

Context keys the engine reads

Key	Values	Effect
`language`	`FRENCH ENGLISH ARABIC DARIJA ARABIZI`	reply language and language policy. Case-insensitive; anything else = unset, and hint/explain detect it from the text.
`level`	free text (`"beginner"...`)	injected into prompts; part of the cache key.
`style`	free text (`"analogy"...`)	explain style hint; part of the cache key.
`confusion`	`LOW MEDIUM HIGH`	explain opener (pass classify's value).
`difficulty`	1–3	level of the practice exercise appended to explain (default 1).
`escalate`	bool	keep explain on mid-size even when quota is low.
`complexity`	1–5	`>= 4` counts as escalation (pass classify's value).
`topic`	string	retrieval filter on the docs' `topic` field.
`turn_id`	string	copied to every metrics record of the call (requests per learner turn).

3.3 `Result`

Field	Type	Meaning
`text`	`str | None`	what to show (hint, explain, quiz). `None` for pure JSON tasks.
`data`	`dict | None`	structured output (classify, grade, quiz, explain parts). Already validated.
`model_class_used`	`str | None`	`"compact"`, `"midsize"` or `None` (answered by rules, bank or sympy).
`model`	`str | None`	concrete model name that answered.
`from_cache`	`bool`	served from the exact cache (0 tokens).
`degraded`	`bool`	something was not ideal; read `degrade_reason` (§5).
`degrade_reason`	`str | None`	the main reason; the full chain is in `notes["degrade_reasons"]`.
`eta_seconds`	`float | None`	when retrying makes sense (busy / quota cases).
`mode`	`str | None`	`"fast_extractive"` (course excerpts instead of generated text), `"bank"` (quiz).
`followup_available`	`bool`	a full explanation can be requested later (fast mode).
`events`	`list[Event]`	progress events, each `name`, `t_ms` since start, `data`.
`latency_ms, tokens_in, tokens_out`		cost of this call (nested compress/repair included).
`repaired / safe_default`	`bool`	JSON needed the repair call / fell back to the safe default.
`route_reason`	`str`	why this model class was chosen.
`notes`	`dict`	diagnostics: retrieved chunks, budget, verified formulas, guards…
`request_id, task`	`str`	identifiers (`request_id` matches the metrics records).

Rule of thumb for the feature layer Always show `result.text` (it is never empty for a text task). Branch on `degraded` only for extra UX: show "simplified answer" when `mode == "fast_extractive"`, and offer "try again in `eta_seconds` s" for `service_busy` / `quota_exhausted`. JSON tasks always return a usable `data` shape, even when degraded.

3.4 Events

Events replace token streaming (the platform returns whole responses). They arrive through `on_event` as they happen and are also kept in `result.events`.

Event	Emitted when…
`answered_by_rules`	a rule answered with 0 requests (greeting, stored hint, sympy grade).
`picking_exercise`	quiz is choosing from the bank.
`retrieving, expanding_query`	explain searches the course; glossary terms added.
`no_relevant_context`	nothing passed the relevance gate: "not in the course" answer.
`checking_cache, cache_hit`	exact-cache lookup / hit.
`negative_cache_hit`	the same request failed a check < 10 min ago: fast mode, 0 requests.
`waiting_identical_request`	the same request is already in flight: waiting for its answer.
`partial_result`	explain: up to 3 course sentences, verbatim, before the model call (show them!).
`compressing`	retrieved chunks do not fit the budget: extractive compression.
`queued (position, eta_s)`	waiting for a rate token / concurrency slot.
`generating (attempt, model_class)`	a provider request is being sent.
`wait_stated_delay, retry_after_interval, wait_slot, backoff, fix_request`	a retry action (§7.3).
`trimming_input`	prompt over the input budget: trimmed before sending.
`falling_back_to_compact, breaker_open`	mid-size unavailable.
`fast_extractive`	serving course excerpts instead of a generated explanation.
`repairing_json, using_safe_default`	JSON repair call / safe default.
`guarded`	an output guard fired (reason in `data`).
`gave_up`	the retry layer stopped; a degraded result follows.

4 Tasks: inputs and outputs

Task	Class	Kind	0-request paths	Output
classify	compact	JSON	bare greetings/thanks	`data`: intent, language, confusion, complexity, confidence
hint	compact	text	stored bank hints	`text`: one guiding question
explain	mid-size	text, RAG	cache, negative cache, not-found, fast mode	`text` + `data`: body, opener, exercise
grade	compact	JSON	sympy equivalence	`data`: verdict, mistake, feedback, graded_by
quiz	— (bank)	local	always (template intro)	`text` + `data.exercises`
compress	compact	JSON	lexical fallback	`data.sentences` (used inside explain)

Table 1: The task table. Model class comes from `router/policy.py`; it is the only place that decides it.

4.1 classify

Inputs: `message` (required). **Output** (`data`, validated with pydantic):

```
{"intent": "EXPLAIN|HINT|QUIZ|ANSWER_ATTEMPT|DIAGNOSTIC|OFF_TOPIC|CHITCHAT",
 "language": "DARIJA|ARABIZI|FRENCH|ARABIC|ENGLISH",
 "confusion": "LOW|MEDIUM|HIGH", "complexity": 1-5, "confidence": 0.0-0.9}
```

What happens around the single compact JSON call:

- **Shortcut:** an exact greeting/thanks ("salut", "aslema", "merci"…) → CHITCHAT with confidence 1.0, 0 requests.
- **Normalization, not repair:** code fences, lower-case enums, `"answer attempt"`, `"4.0"` are accepted without an extra call.
- **Repair:** invalid JSON → one compact repair call at temperature 0 (the model sees its own bad output + the error), then a **safe default** (`EXPLAIN`, `MEDIUM`, language from rules, confidence 0).
- **Rules layer:** the language is reconciled with the script and Tunisian dialect markers; model confidence is capped at 0.9 (1.0 = rules); explicit "I don't understand" markers raise confusion to HIGH. Raw model values stay in `notes`.

From intent to task. `engine.router.follow_up(classification)` returns the next task, with one override: a learner with HIGH confusion who asks for a quiz or a level test gets an explanation first.

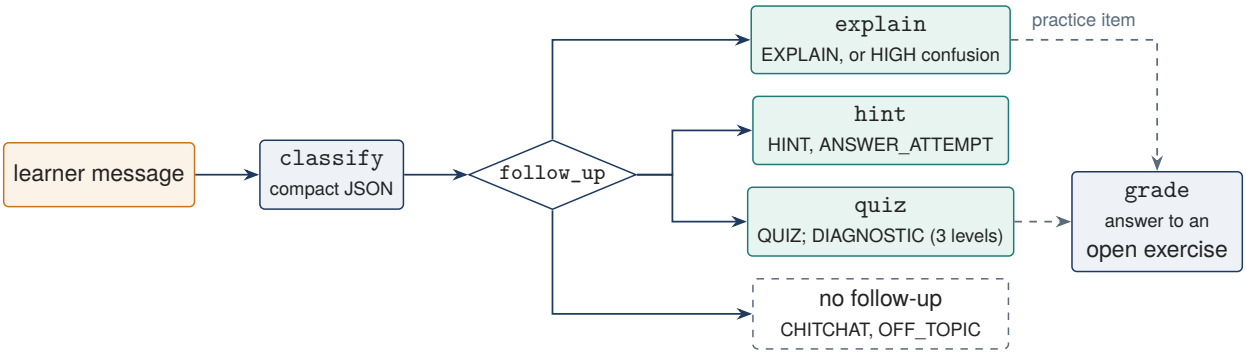


Figure 2: One learner turn, as `scripts/chat.py` implements it. Your feature layer owns this loop.

4.2 hint

Inputs: `problem` (required); `attempt`, `solution` (used only by the leak guard, never sent as an answer); for bank exercises `stored_hints` (list) + `hint_level` (1-based). **Output:** `text`, one guiding question (prompt asks ≤ 25 words).

With `stored_hints`, the hint is served from the bank in order: 0 requests, `data = {hint_level, hints_total, source: "bank"}`. Otherwise one compact call, then guards in order: repetition loop → template; > 40 words → keep the leading complete question or template; solution leaked (verbatim or as "= value") → template; wrong language → template; foreign Latin words in an Arabic-script hint → flagged. Every guarded hint is `degraded` and never cached.

Real example: `problem="Calcule la dérivée de f(x) = 3x² + 5x"`, `attempt="6x"` → *"Qu'est-ce qui se passe lorsque tu multiplies la dérivée de 3x par 2, puis ajoutes 5 ?"* (compact, 4.0 s, 225/30 tokens). Hint quality depends on the compact model; the guards only catch the unusable cases.

4.3 explain

Inputs: `question` (required); `history` (list of `{"role","content"}` turns, kept newest-first within budget); `exclude_exercises` (ids already served). **Context:** `language`, `confusion`, `difficulty`, `escalate/complexity`, `topic`.

Output: `text` = opener + body + practice line; `data = {body, opener, exercise}`, where `exercise = {id, concept, level, statement, expression}` (no solution, no hints). In fast mode: `data = {framing, excerpts: [{chunk_id, text}], exercise}` and `mode="fast_extractive"`.

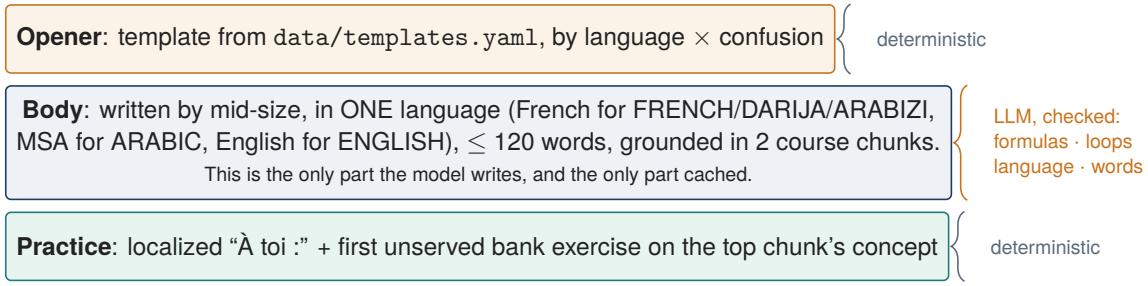


Figure 3: Composition of an explain answer: narrow what the LLM writes, check what it does write.

Outcomes you can get from explain:

Situation	What the learner gets
normal	opener + mid-size body + exercise; cached for 7 days.
nothing in the course passes the gate	localized "not in the course material" text; 0 requests, not degraded, not cached.
mid-size busy, breaker open or quota low	**fast mode**: up to 5 course sentences verbatim + framing line + exercise; degraded, never cached.
body fails a check	text kept but `degraded` (`ungrounded_formula`, `wrong_math`, `language_drift`, `contamination`); not cached; failed checks also set a negative marker.
body is a repetition loop	≥ 40 good words before the loop → truncated text kept (`loop_truncated`); otherwise fast mode (`degenerate`).

4.4 grade

Inputs: `problem`, `solution`, `attempt` (all required); `common_mistakes` as a list of dicts with keys `expr` and `feedback`, or of `[expr, feedback]` pairs. **Output:** `data = {verdict: CORRECT|INCORRECT|PARTIAL|UNKNOWN, mistake, feedback, graded_by: "sympy"|"llm"}`.

If both answer and solution parse as math (`x^2`, x^2, `2x`, `x(x+3)`, $\sqrt{x}$, French decimal commas, an `f'(x) =` prefix…), `sympy` decides by `simplify(a - b) == 0`: 0 requests. A wrong answer equivalent to a stored mistake gets that mistake's feedback. Otherwise (sentences, text solutions) one compact JSON call grades it; feedback that would leak the solution is blanked. `UNKNOWN` is the safe default when the LLM grader fails.

Real example: `solution="f'(x) = 4x^3"`, `attempt="x^3"`, mistake `"x^3"` → `INCORRECT`, `mistake="x^3"`, feedback from the bank, `graded_by="sympy"`, 81 ms, 0 tokens.

> **Security** `sympy`'s parser evaluates Python. Learner text is reduced to a whitelist (digits, `x`, operators, brackets, `sqrt ln exp sin cos tan`; ≤ 80 characters) before parsing, and

anything else goes to the LLM grader. Keep that whitelist if you extend the math parser (`engine/mathcheck.py`).

4.5 quiz

Inputs (all optional): `message` (the learner's request: its concepts are matched through the glossary), `concept`, `difficulty` (1–3, nearest level is used), `exclude` (ids already served), `diagnostic=True` (3 exercises: levels 1/2/3 on three concepts), `intro` (`True` = template, `"llm"` = compact-written cached line, `False` = none). **Output:** `text` (intro + statement), `data = {exercises: [...], requested_concepts, diagnostic, intro}`, `mode="bank"`. Always 0 requests unless `intro="llm"`.

To grade the learner's answer, look up the full exercise with `engine.exercises.get(id)` (it has `solution`, `hints`, `common_mistakes`) and call `grade` / `hint` with them, as `scripts/chat.py` does.

4.6 compress

Used internally by explain when the retrieved chunks do not fit the prompt budget. Inputs: `question`, `chunks`. The model sees numbered sentences and returns `{"keep": [numbers]}`, so it cannot invent text (10 output tokens). If that fails, a lexical pick is used. A chunk that loses all its sentences gets its best one back: compression never drops a relevant chunk.

5 Degrade reasons: the complete catalog

degrade_reason	Meaning	Feature-layer suggestion
`quota_low`	local quota tokens < `QUOTA_LOW_THRESHOLD`: explain sent to fast mode proactively	show excerpts; offer full answer later
`breaker_open`	mid-size had 3 saturations in 60 s: skipped for 30 s	same
`midsize_saturated`	mid-size refused after retries: runtime fallback	same
`service_busy`	every class saturated / concurrency refused (`eta_seconds` set)	"busy, retry in N s"
`quota_exhausted`	stated quota delay longer than `MAX_RETRY_WAIT_S` (BACKGROUND: `_BACKGROUND`); `eta_seconds` = the remaining block	same
`input_truncated`	the provider cut the start of the prompt (`prompt_tokens` far below the prompt size); never cached	show a fallback; shorten the input
`service_unavailable`	transient 5xx/network errors exhausted (`eta_seconds` set)	same
`request_too_large`	prompt cannot be trimmed under the ceiling	shorten history/input
`provider_error`	other 4xx (bad key, unknown model…)	alert; check config

degrade_reason	Meaning	Feature-layer suggestion
midsize_unavailable	mid-size timed out or is down; eta_seconds = when it is probed again (v1.2)	fast mode text is in the result
deadline	explain had no body after 25 s; the generation continues into the cache (v1.2)	fast mode text now; the next ask is a hit
superseded	the same learner queued a newer INTERACTIVE request (v1.3)	drop this result silently
room_busy	another AI request of the room is running; eta_seconds (v1.3)	disable the button, retry after the ETA
invalid_image	ocr_transcribe: not a readable image (v1.3)	ask for another photo
low_confidence	ocr_transcribe: text returned with visible reading problems (v1.3)	show with confidence_note
json_invalid	check_claim: no valid JSON after the repair call (v1.3)	treat the verdict as unknown
ungrounded_formula	a formula neither verifiable nor in the sources	show; mark "check with teacher"
wrong_math	a derivative/identity claim sympy proves false	consider hiding the body
explain_guard:loop_truncated	repetition loop cut after the last full sentence	show
explain_guard:degenerate / :empty	unusable output: served in fast mode	show excerpts
explain_guard:language_drift	body in the wrong script	show; maybe retry later
explain_guard:contamination	foreign Latin words in an Arabic-script answer	show
hint_guard:degenerate / :too_long / :answer_leak / :language_drift	hint replaced by a safe template	show (the template is fine)
hint_guard:contamination	flagged words, text kept	show

Degraded results are never cached. The deterministic-check failures (ungrounded_formula, wrong_math, loop_truncated, degenerate) also set a **negative marker** so that the same question within 10 minutes does not burn another identical request (§9).

6 Inside `engine.run`: the request lifecycle

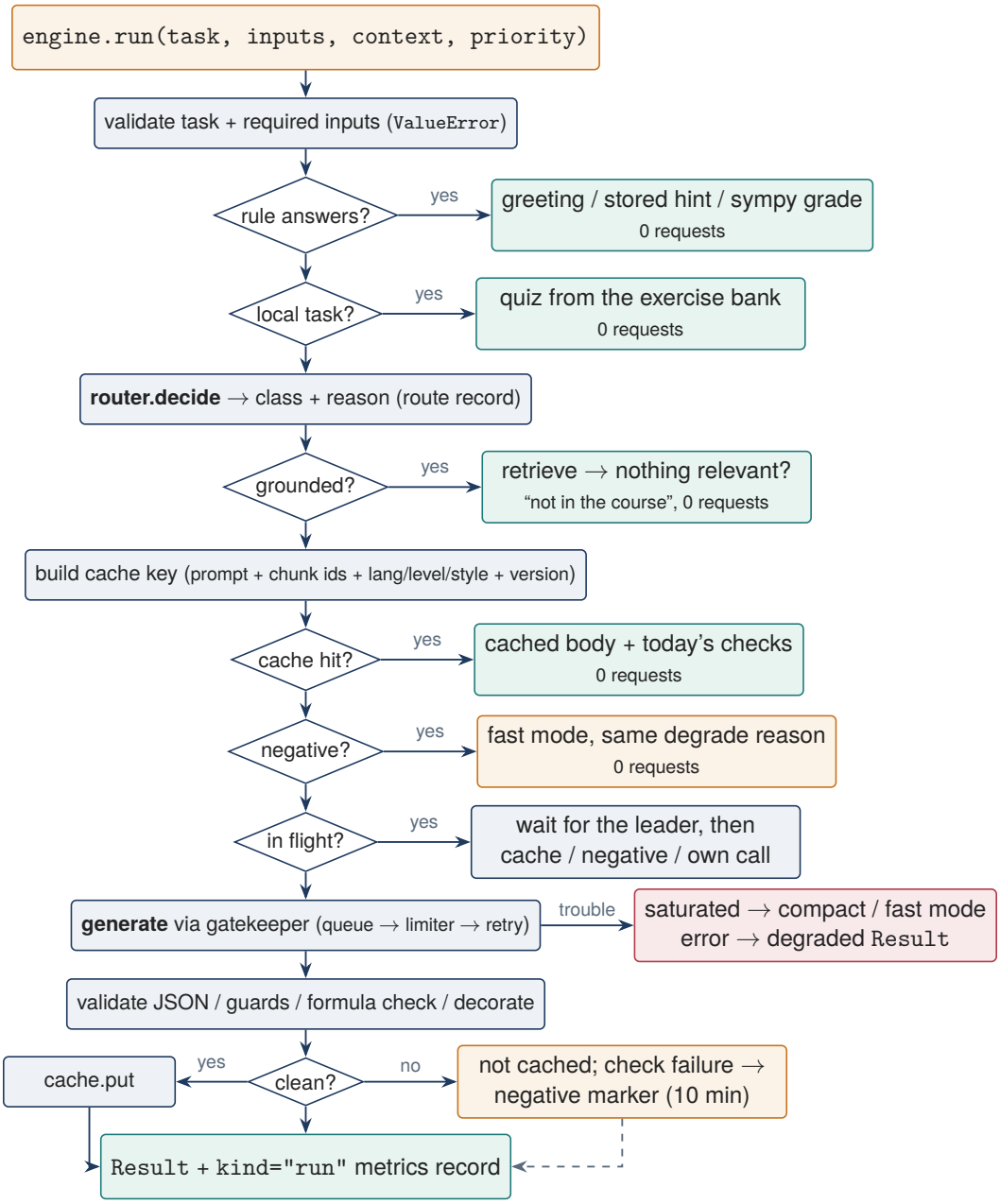


Figure 4: What `engine.run` does, in order (`engine/core.py`). Every right-hand exit ends in the same `Result` shape.

Two details matter when you modify the flow:

- **The cache key uses the router's preferred class**, not the class actually used. A request moved to compact by low quota can still be served a cached mid-size answer, which is better than a fresh compact one.
- **For explain, the key is built from the chunk ids**, not from the compressed text, so the lookup happens before compression and a hit costs nothing, compression included.

7 Gatekeeper: living inside the platform limits

Every LLM and embedding request goes through `gatekeeper/`. Nothing else talks to the provider; this is what makes the metrics trustworthy.

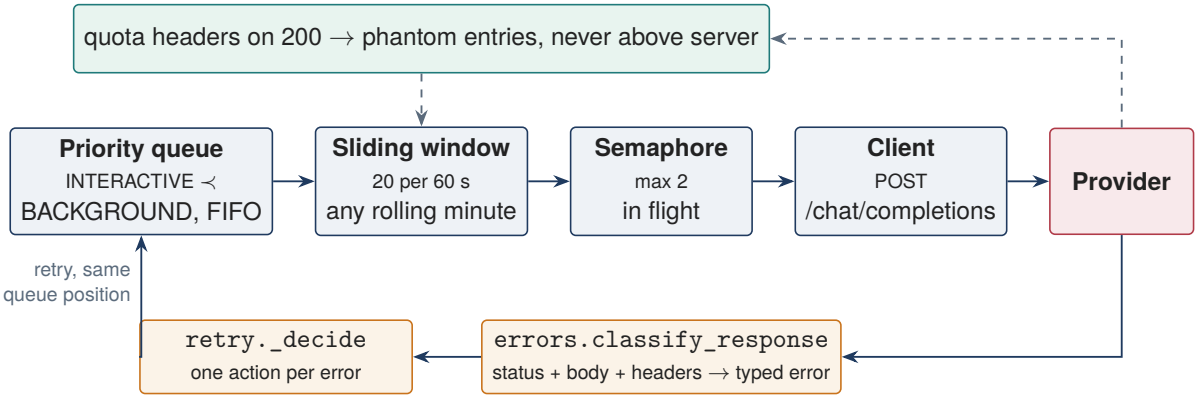


Figure 5: The gatekeeper path of one attempt. Quota waits happen in the window and never hold a concurrency slot.

7.1 Rate and concurrency are two mechanisms

- **Sliding window** (rate): a log of send times; at most `RATE_LIMIT_PER_MIN` (20) sends in any rolling `RATE_WINDOW_S` (60) seconds. FelCloud keeps the key closed for the full minute after the 21st request, so a bucket with a burst reserve overshoots it. Embeddings have their own window (60/min), because the platform has a separate embeddings quota.
- **Semaphore** (concurrency): at most `MAX_CONCURRENT` (2) requests in flight, shared by chat and embeddings (assumed per key).
- A request waits for a free window slot *before* taking a concurrency slot, and records its send only once it has the slot. So a quota wait never blocks a slot, and requests queued on the semaphore cannot burst past the window.
- **Server feedback**: `x-ratelimit-api_key-remaining-requests` on every 200 adds phantom entries when it is below the local free count, so local never exceeds the server; a stated quota delay blocks the window until it expires.

7.2 Priority queue

One dispatcher per kind (chat, embeddings) waits until the limiter has capacity and only *then* pops the best waiter. Choosing at grant time is what lets an interactive request that arrives behind 50 background ones leave first. A retry keeps its original sequence number, so a request that waited out a quota delay is not sent to the back. The queue also keeps a moving average of how long requests hold a slot, which gives the `eta_s` of `queued` events and of busy results:

$$\text{eta} \;=\; \text{Retry-After} \;+\; \frac{(\text{queue depth} + 1) \times \overline{t_{\text{slot}}}}{\texttt{MAX_CONCURRENT}}$$

7.3 The platform contract as code

Provider says	Typed error	Action	Event
429 quota (rate_limit_type: requests, retry-after: 60; any 429 that is not cooldown/concurrency)	QuotaExhausted	delay = the retry-after header only; the window is blocked until it expires; BACKGROUND waits up to 65 s, INTERACTIVE up to 30 s, otherwise give up with eta = the remaining block; never hop models (the quota is per key)	wait_stated_delay
503 + Retry-After, or LiteLLM 429 all_deployments_in_cooldown	ModelSaturated	mid-size: retry once, then the router falls back. compact: up to 3 retries	retry_after_interval
429 parallel/concurrent (rate_limit_type: max_parallel_requests)	ConcurrencyLimited	wait min(Retry-After, CONCURRENCY_RETRY_WAIT_S) = 2 s (it states 60 s)	wait_slot
400 naming a ceiling	CeilingExceeded	fix once (trim max_tokens or the longest message), retry once	fix_request
5xx, network, timeout	TransientError	exponential backoff + full jitter (only when no delay is stated)	backoff
other 4xx	FatalError	give up immediately	gave_up
200	-	read quota headers, pace	-

Table 3: gatekeeper/errors.py (mapping) and gatekeeper/retry.py (actions). At most MAX_ATTEMPTS=4 attempts.

What was observed on the real FelCloud (recon, 22 Oct) FelCloud is a LiteLLM proxy behind Caddy (server timeout 90 s → client timeout 85 s). Saturation arrived as **429 all_deployments_in_cooldown** with Retry-After: 5, not the annex's 503, so both are mapped to ModelSaturated. There was **no 400 for long prompts**: input was **silently truncated near 2048 tokens**. The client therefore enforces its own input budget (INPUT_TOKEN_BUDGET=1800 then) and trims *before* sending. (superseded in v1.1: truncation starts near 4096 tokens, not 2048, and the budget is now 2800; see "Measured behaviour)

Updated by the probes of 26 Oct (engine v1.1) Both 429 refusals were observed: status 429, retry-after: 60, rate_limit_type requests or max_parallel_requests; Limit resets at in the body is the refusal time plus 60 s and is ignored. Both are CONFIRMED(felcloud) in errors.py, with the real responses as test fixtures. The input window is about **4,096 tokens**, and overflow silently drops the *start* of the prompt, so the engine checks estimated prompt + output cap against CONTEXT_WINDOW × 0.85, estimates at 2.3 characters per token (Arabizi, the worst measured case), raises INPUT_TOKEN_BUDGET to 2800, and degrades any answer whose prompt_tokens shows the input was cut (input_truncated). max_tokens is ignored; only a top-level options.num_predict caps the output, and finish_reason stays stop when it cuts, so completion_tokens ≥ cap means cut.

8 Router: compact first, mid-size by rule

router/policy.py holds the routing table (ROUTES) and every routing decision; each one is logged as a kind="route" metrics record with its reason.

```
ROUTES = {"classify": Route(C), "hint": Route(C), "grade": Route(C),
          "compress": Route(C), "summary": Route(C), "quiz": Route(C),
          "explain": Route(M, fallback=C)}
```

For explain the decision is, in order:

1. default mid-size;
2. if local quota tokens < `QUOTA_LOW_THRESHOLD` (2) and the call is **not escalated** (`escalate=True` or `complexity >= 4`) → compact, `degraded`, reason `quota_low`;
3. if the mid-size circuit breaker is open → compact, reason `breaker_open` (escalation does not bypass it: escalating cannot help a class that is down);
4. at run time, if mid-size stays saturated after its retry → `midsize_saturated` fallback.

For explain, "compact" means **fast mode**: compact does not write the explanation (it misquoted the course in tests); the learner gets verbatim course sentences plus one framing line.

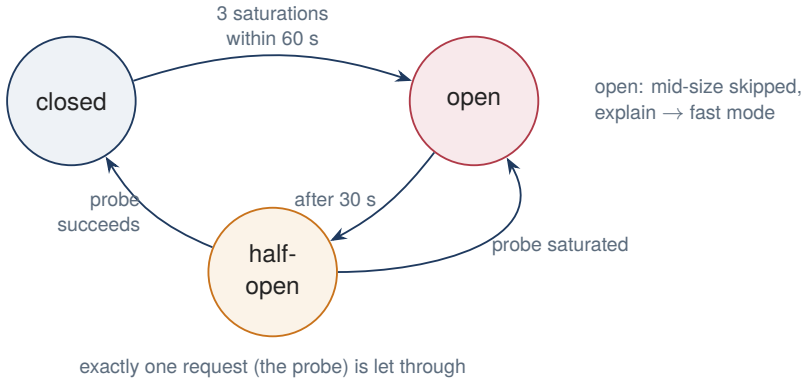


Figure 6: Mid-size circuit breaker (`router/breaker.py`). Only saturation responses count; quota refusals are per key and say nothing about class load.

A probe that never reaches mid-size (cache hit, not found, fast path) releases its slot at the end of `engine.run`; a probe that never reports expires after one cooldown, so the breaker cannot stay stuck half-open.

9 Caching and deduplication

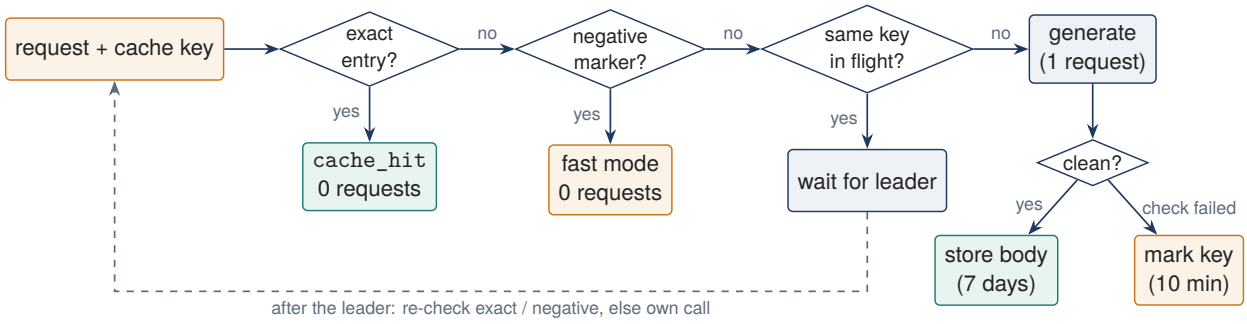


Figure 7: Three ways the engine avoids a repeated identical request.

- **Exact cache** (SQLite, `CACHE_TTL_DAYS=7`). Key = SHA-256 of task, preferred class, normalized prompt (whitespace collapsed, Unicode NFC, *no* lower-casing: `X` ≠ `x` in math), language,

level, style, prompt-file version, and for explain the retrieved chunk ids. Editing a prompt bumps its version and invalidates its old answers automatically.

- **Only clean results are stored**: `put()` itself refuses degraded, fallback and safe-default values. Cache hits re-run today's deterministic checks (an explain entry that now fails is deleted; classify re-applies its rules).
- **Coalescing**: a request whose key is already being generated waits for the leader and is then served from the cache (`waiting_identical_request` → `cache_hit`).
- **Negative cache** (`NEGATIVE_CACHE_TTL_S`=600, 0 = off): an explain answer that failed a deterministic check marks its key. A repeat within the TTL gets fast mode with the same degrade reason and **0 requests**, logged as `outcome="negative_cache_hit"`, `from_negative_cache=true`. Transient failures (saturation, quota) are never marked.
- **No semantic cache**: it was specified for explain, but FelCloud exposes no embedding model, so it was not built (§17).

10 Retrieval (RAG) for explain

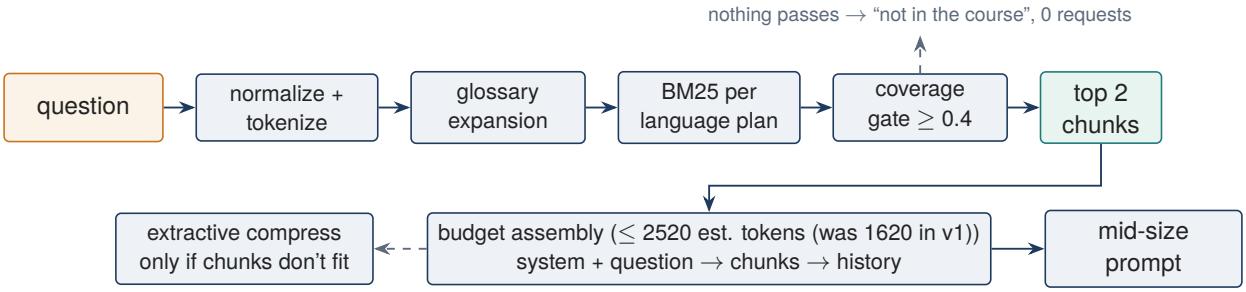


Figure 8: Retrieval pipeline. No embeddings and no model call: BM25 in pure Python.

Course documents

`data/docs/*.md`, one file per topic and language, with a front matter (`topic`, `level`, `language`, `title`, optional `status: draft`) and `##` sections of about 250–300 tokens, each tagged `<!-- concept: id -->`. `python -m scripts.index_docs` writes `data/index/chunks.json`; chunk id = `<file stem>#<section slug>`, stable across re-indexing. Drafts are skipped until reviewed.

Tokenization

NFKC (x² → x2); Arabic diacritics/tatweel stripped and letters normalized before splitting; accent folding and light plural/*e* stemming on Latin words only (`dérivée`, `derivee`, `dérivés` → `deriv`); Arabic prefix stripping; FR/EN/Arabizi/Arabic stopwords.

Glossary

`data/glossary.yaml` maps 327 learner variants (Darija, Arabizi, MSA, English) to 60 French course terms, deterministically. Course terms are weighted 0.5 so the learner's own words dominate. This is what lets "mochtaqa" or an Arabic-script question find a French chunk.

Relevance gate

coverage, not BM25 score: the IDF-weighted share of the question's concepts found in the chunk. On a one-topic corpus raw scores fail ("the function in my code is slow" outscored a real question). Measured: on-topic ≥ 0.49, off-topic ≤ 0.28 → threshold 0.4.

Language plan

DARIJA/ARABIZI → French chunks; ARABIC → Arabic, else French; ENGLISH → English, else French; FRENCH → all.

Budget
prompts are assembled within `INPUT_TOKEN_BUDGET × (1 − 0.1)` = 1620 estimated tokens; history is dropped oldest-first without gaps; everything dropped is listed in `notes.dropped`. (superseded in v1.1: `INPUT_TOKEN_BUDGET` is 2800, so 2520 estimated tokens; the 1620 above is the v1 figure.)

Swap the retriever `RETRIEVER=embedding` selects `EmbeddingRetriever` (cosine over gatekeeper embeddings, using the separate embeddings quota). Any object with `async index(chunks)` and `async search(question, k, filters, expansion) -> list[Hit]` can be passed to `Engine.create(retriever=...)`.

11 The deterministic layer

The design principle: **narrow what the LLM writes, and check what it does write.** Small models on CPU are slow and unreliable; every sentence a rule or a template can produce is a request saved and a failure avoided.

Mechanism	What it does
Templates (`data/templates.yaml`)	openers by language × confusion, "your turn" lines, fast-mode framing and quiz intros (Darija/Arabizi always from templates), generic grading feedback.
Exercise bank (`data/exercises/*.yaml`)	15 exercises, 11 concepts, levels 1–3, each with 3 graded hints and common mistakes; statements rendered in FR/EN/AR from phrase templates.
Formula check (`engine/grounding.py`)	every "A = B" and every formula-like run in an explain body is *verified* (sympy: `f'(x) = D` against the answer's own `f(x) = E`, values, identities), *skipped* (assignments like `n = 2`, equations to solve, refuted claims after "not"), or else must appear in the sources. False → `wrong_math`; neither → `ungrounded_formula`.
Loop detection (`engine/tasks.py`)	a 3-word phrase recurring 3+ times in a row, each within 12 words: salvage the text before it if ≥ 40 words and ending on a full sentence.
Language checks	classify's language reconciled with script and Tunisian markers; explain/hint drift guards; Latin-word contamination in Arabic-script answers.

12 Metrics: what is measured

Every provider attempt, routing decision, breaker change, cache hit and engine call writes one `CallRecord` (`metrics/record.py`) to `METRICS_PATH` (JSONL).

Field	Meaning
`kind`	`chat` / `embed` (one attempt or cache hit), `route`, `run`, `breaker`
`request_id, turn_id, attempt`	one engine call (shared by its retries) / one learner turn / attempt number
`task, priority, purpose`	task name; INTERACTIVE/BACKGROUND; `repair`, `framing` for side calls
`model_class, model`	what was called
`request_hash`	hash of the full request body: the same hash twice = repeated identical request
`status, outcome, action`	HTTP status; `ok`, `quota_exhausted`, `saturated`, `cache_hit`, `negative_cache_hit`…; what was done about it
`wait_ms, latency_ms`	time waiting (limiter, delays) / provider time
`tokens_in, tokens_out, quota_remaining`	usage and the quota header
`from_cache, from_negative_cache, degraded, degrade_reason, mode, graded_by, eta_s, detail`	run outcome details; `detail` = route reason or provider message (never secrets)

```
python -m scripts.load_test --sim-profile normal --n 50       # also: saturated,
    quota_low, capacity
python -m scripts.metrics_report metrics/load_normal.jsonl metrics/load_saturated.
    jsonl --out reports/x
```

The report gives, side by side: provider requests, calls per model, compact/mid-size split, tokens, cache hit rate, repeated identical requests, refusals and how each was handled, degradations, breaker trips, latency percentiles per task and priority, and a per-turn capacity estimate.

13 What is new in v1.3

v1.3 makes the engine ready for every Phase 1 feature of the team's contracts document. The public interface is unchanged (`engine.run`, `Result`, `Event`); everything below is additive. The contract the feature layer codes against is `docs/ENGINE_API.md`: every task, its inputs and outputs, events, degrade reasons, request cost and a JSON example per outcome. It is generated from the engine's real output (`python -m scripts.gen_api_examples -doc`), and a test fails when it drifts from the code. Until the real engine is wired, `engine.fake.FakeEngine` has the same `run` signature and returns the same shapes, events and timings in three modes (normal, slow, degraded), with `context["fake_outcome"]` to force one state, so the UIs can show every state without any LLM.

13.1 Identity, fairness and the background reserve

- **Ids.** `learner_id`, `room_id`, `course_id` and `class_id` in the context are copied into every metrics record of the run (ids only, never names), also those of helper runs and of the generation that outlives an interactive deadline. They are not part of any cache key.
- **One waiting request per learner.** A newer INTERACTIVE request of the same learner replaces the older one while it still waits in the queue; the older returns at once, degraded `superseded`, nothing sent. A request already running is never touched; BACKGROUND work is not limited.
- **One request per room.** A second `run` with the same `room_id` while one is in flight returns `room_busy` with an `eta_seconds` (an average of the room's recent run times, minus the time already spent). Helper runs (the grade call inside `check_answer`) keep the learner's ids but not

the room.

- **Background reserve.** A BACKGROUND chat request is not granted while fewer than `BACKGROUND_RESERVE` (6) sends remain in the 20-per-minute window; it waits for entries to expire and announces it once (event `waiting_reserve`). A prep plan or an ingest can therefore never use the window a learner is about to need. Interactive requests ignore the reserve.

13.2 The vision class

`ModelClass.VISION` (`MODEL_VISION`, default `deepseek-ocr`) reads images. The client sends OpenAI content parts (an `image_url` with a base64 data URL); the engine decodes, orients and resizes every image to at most one megapixel and counts its estimated tokens in the input budget (a request that cannot fit is refused locally, nothing sent). Vision has its own timeout (`TIMEOUT_VISION_S`, 40 s) and, like mid-size, is never retried after a hang. It shares the chat rate window and the two concurrency slots with the other classes until the organizers say otherwise; mid-size and vision together may hold at most `MAX_CONCURRENT`-1 slots, so compact work always has one (an interactive photo can therefore wait behind a mid-size generation that is still running into the cache; its `queued` event carries an ETA). A learner's explain is bounded by the 25 s deadline in every path, also while it waits for an identical request that another learner or a prep page is generating. Whether a vision request really counts in the 20-per-minute window is measured by `scripts/load_mixed -real -probe-vision` (not yet run: no slot).

13.3 New tasks

task	class	requests	what it does
`ocr_transcribe`	vision	1 (0 cached)	photo of work → {text, latex, confidence_ note}; never grades; cache by image content
`check_claim`	rules, compact	0 or 1	is this claim true in the course: sympy for formulas, retrieval gate (`NOT_IN_COURSE`), else one JSON call
`check_answer`	rules, compact	0 to 2	a worked solution line by line: OK, WRONG or UNCHECKED; stored mistakes, buggy rules, guarded guiding question; the last line through grade
`prep_plan`	none	0	study plan: prerequisites first, lowest mastery next, spread at the learner's pace, mastered concepts reviewed
`generate_page`	mid-size	0 or 1	a prep page: explain at BACKGROUND through the tutor's cache, a bank worked example, three quiz items (variants fill gaps)

Three rules that run through them: a line or text that cannot be parsed is `UNCHECKED`, never `WRONG`; a guiding question is checked so it never states the answer (the hint guard plus the true derivative written out); and a verdict that sympy can compute is never left to a model. `check_claim` decides in this order: a refuted formula is `CONTRADICTED` with the correct form (half refuted, half verified: `PARTLY`); a verified formula with no other claim around it is `SUPPORTED`; anything else goes to retrieval, whose gate gives `NOT_IN_COURSE` at no cost, and only then to one compact JSON call with the repair ladder and a degraded `PARTLY` as the safe default. `explain` gains two inputs: `passage` (the text a learner selected: in the prompt, and retrieval runs on it first) and `board_text` (whiteboard text, trimmed to a share of the budget); both are in the cache key by a hash of the full text.

13.4 Course content

A course is a graph (`data/courses/<course>.yaml`: units with order and prerequisites, concepts with prerequisites and related concepts, titles in French, English and Arabic). Concept ids are those of the exercise bank and of the `<!- concept: id ->` tags of the course documents, and a test checks the three agree in both directions and that `requires` has no cycle. The retriever can add and remove chunks at runtime (`Engine.add_material`, `remove_material`), persists them per course, and filters by `course_id` plus visibility: public course documents, class material for that `class_id`, private uploads for that `learner_id`; a viewer without an id never sees class or private material. Professor material has its BM25 score multiplied by `SOURCE_BOOST_PROFESSOR` and wins ties; the coverage gate is untouched, so the boost reorders relevant chunks but never lets an off-topic one through. `rag/ingest.py` turns a PDF (PyMuPDF), a DOCX or a Markdown file into chunks split by headings, tags each chunk's concept by BM25 against the concept titles and the glossary with a confidence (low confidence is what a professor confirms), and flags pages without text as `needs_ocr`; `Engine.ingest_file` reads up to 20 of those pages per file through `ocr_transcribe` at BACKGROUND, one at a time.

13.5 Cache administration and scrubbing

`engine.cache.invalidate(request_id)` removes the entries behind a request's answer, whether that request generated the entry or was served it (a professor rejecting a flagged answer), and a request id names one generation of an entry. Provider error messages are scrubbed of the API-key hash the proxy echoes (`gatekeeper/errors.py`), so it never reaches a metric or a log.

13.6 Verification status

Everything above has unit tests with a fake transport and virtual time (no LLM, no Ollama). The `check_claim` accuracy on the 30-claim set, the one-OCR-call smoke, the measurement of vision against the rate window and the mixed load on the live API need a FelCloud slot and have not run; the mixed-load scenario has run in virtual time through the constraint simulator (`reports/load-mixed-virtual-v13.md`), which tests the engine's scheduling and not the models. PyMuPDF could not be installed offline, so the PDF reader is tested through a stand-in module only.

14 Measured behaviour (engine v1 local, v1.1 on FelCloud)

Load tests run real Ollama models (llama3.2:3b compact, llama3.1:8b mid-size, on CPU) behind the constraint simulator: 50 engine calls per profile at 30 arrivals/min, or 40 learner turns for *capacity*. *saturated* = 90 % mid-size and 20 % compact saturation; *quota_low* = the provider really allows 10/min with burst 4 while the client believes 20/min.

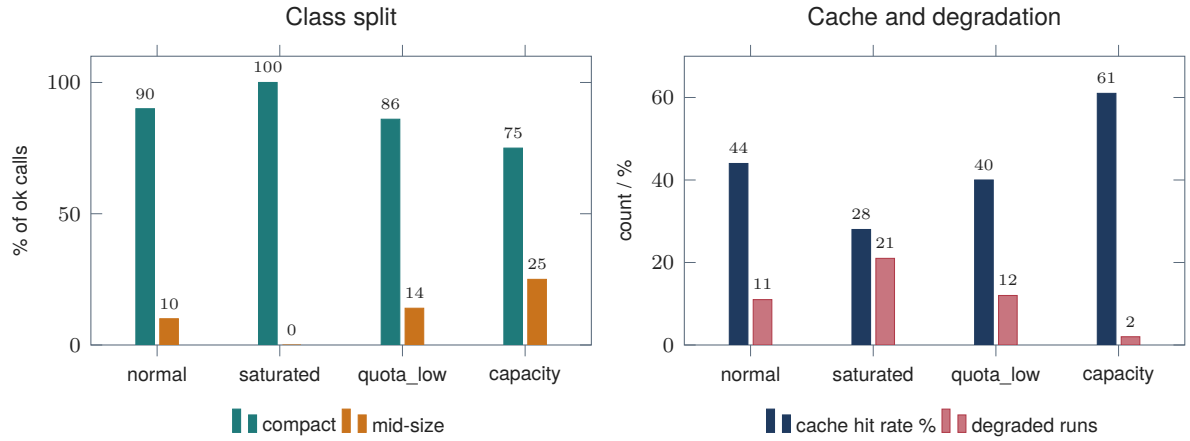


Figure 9: `reports/load-engine-v1.md`. 0 exceptions reached the caller in any profile; all 7 quota refusals in *quota_low* were handled by waiting the stated delay.

	normal	saturated	quota_low	capacity
engine calls	50	50	50	69
provider requests (all attempts / ok)	28 / 21	35 / 21	33 / 21	27 / 24
refused attempts	7	14	12	3
explain served in fast mode / generated	11 / 10	21 / 0	12 / 9	1 / 10
breaker trips	1	1	1	1
input tokens per ok call	455	427	470	619
repeated identical provider requests	0	0	0	$1 \rightarrow 0^*$

Table 4: Load test details. *the one repeat was a degraded explain re-sent; fixed by the negative cache (replay test; a live rerun had 0 repeats).

Capacity. A learner turn costs 1.73 engine calls, and **2.00 provider requests when nothing is cached**. At 20 requests/min per key that is about 20 active learners sending one message every 2 minutes, 10 at one per minute. Locally the binding limit is concurrency, not quota (11.5 s mean latency $\rightarrow$ 10.4 req/min); on FelCloud (v1.1, see below) the quota binds instead.

14.1 The same measurements on FelCloud (engine v1.1, 26 Oct)

Real provider, simulator off (`reports/felcloud-capacity-quality-2026-10-26.md`); the local column is the table above (Ollama behind the simulator). The capacity run is 40 learner turns at about 15 engine calls/min.

	local v1 (Ollama)	**FelCloud v1.1**
engine calls per turn	1.73	1.73
provider requests per turn, observed	0.68	0.57
provider requests per *cold* turn	2.00	1.77
cache hit rate (small question pool)	61 %	62 %
mean provider latency	11.5 s	5.4 s
binding limit	concurrency, 10.4 req/min	quota, 20 req/min (concurrency ceiling 22.3)
cold-turn latency p50 / p95	n/a	10.3 / 16.6 s
active learners, 1 message / 2 min (cold)	20 (quota), 10 (at local latency)	23
active learners, 1 message / min (cold)	10	11
refusals, degraded runs, exceptions	3, 2, 0	0, 0, 0

Table 5: Capacity. Plan on the cold-turn rows: the cache rate comes from a 14-message pool. The quota binds only slightly, so a slower mid-size model moves the limit back to concurrency.

Explain quality	**local (Ollama 8B, 91 answers)**	**FelCloud llama3.1-8b, 35 answers**
new examples	10 (11 %)	3 (9 %)
wrong_math	1	1
ungrounded_formula	2	3 (one also wrong_math)
loop guard	2/50 (4 %)	4/35 (11 %); one reproduced false positive
language drift, contamination, truncated output	n/a	0, 0, 0
output tokens / latency	n/a	mean 110 tokens; p50 9.3 s, p95 12.2 s

Table 6: Of the flagged formulas: 2 real model errors (wrong power rule, an invented muddled example), 1 checker limitation (a cut span), 1 false positive ($f(0) = 1$) whose answer also contains an undetected sign error. One run, temperature 0.3: differences of one or two answers are noise.

Priority and outages. At 40 arrivals/min (n=60) the INTERACTIVE `engine.run` p95 was 26.8 s, made of limiter and queue wait rather than provider time; BACKGROUND absorbed the queueing (about 28 s in the queue against 0.8 s median for interactive). During a mid-size outage (26 Oct, about 40 minutes of timeouts) the breaker never opened on timeouts and two hung explains blocked classify calls for up to 190 s; see `reports/midsize-outage-2026-10-26.md` and the change list there.

Evaluation	**Result**
classify, llama3.2:3b, 15 messages	intent 87 %, language 93 %, p50 2.6 s; manual set 4/4
retrieval (BM25), 15 + 4 off-topic	hit@1 13/15, hit@3 14/15, off-topic 4/4 rejected
cross-language (Darija/Arabizi → French docs)	hit@1 0/8 without glossary → 6/8 with it
explain formula check, 91 answers	10 with new examples (correct), wrong_math 1, ungrounded 2
explain loops, 50 answers	loop rate 4 % (2/50), salvage 1/2

Table 7: Quality evals (`scripts/eval_classify.py`, `eval_retrieval.py`, `eval_explain.py`). The glossary and its test questions were written by the same team: 6/8 is an upper bound until real learner questions are collected.

15 Configuration reference

All settings come from the environment through `config.py`; code never names a model, a URL or a key. Defaults below are the code defaults (`.env.example` documents the same values).

Variable	Default	Meaning
Provider and models		
`LLM_BASE_URL, LLM_API_KEY`	`required`	OpenAI-compatible endpoint and key (never logged)
`MODEL_COMPACT, MODEL_MIDSIZE`	`required`	comma list = in-class preference order
`MODEL_EMBED`	`empty`	embedding model (only for `RETRIEVER=embedding`)
`MODELS_WITHOUT_JSON_MODE`	`empty`	models that break on `response_format`
`REQUEST_TIMEOUT_S`	`85`	FelCloud's server timeout is 90 s
Platform limits (client side)		
`RATE_LIMIT_PER_MIN, RATE_WINDOW_S`	`20, 60`	chat sliding window: at most 20 sends in any 60 s
`RATE_WINDOW_SLACK_S`	`0.5`	a send frees its slot this long after the window
`EMBED_RATE_LIMIT_PER_MIN, EMBED_RATE_WINDOW_S`	`60, 60`	embeddings window (separate quota)
`RATE_BURST, EMBED_BURST`	`20, 60`	deprecated (simulator bucket only)
`MAX_CONCURRENT`	`2`	requests in flight
`BACKGROUND_RESERVE`	`6`	BACKGROUND chat waits while fewer than this many sends remain in the window (v1.3)
`MIDSIZE_MAX_SLOTS, VISION_MAX_SLOTS`	`1, 1`	slots each slow class may hold; together they leave one slot to compact (v1.2, v1.3)
`MODEL_VISION`	`deepseek-ocr`	the vision class (v1.3)
`TIMEOUT_VISION_S`	`40`	vision request timeout; under the server's 90 s
`VISION_MAX_PIXELS, VISION_MAX_BYTES`	`1000000, 12000000`	images are resized to this many pixels; larger files are refused
`IMAGE_TOKENS_PER_MEGAPIXEL`	`1000`	input tokens counted per megapixel of image (an assumption until measured)
`MAX_TOKENS_VISION, OCR_MAX_TOKENS`	`700, 700`	output caps for the vision class and the OCR task
`CHECK_CLAIM_MAX_TOKENS, CHECK_ANSWER_MAX_TOKENS`	`120, 80`	per-task output caps
`COURSES_DIR`	`data/courses`	course graphs (units, concepts, prerequisites)
`RAG_COURSE_INDEX_DIR`	`data/index/courses`	runtime material (uploads, professor documents), one file per course
`SOURCE_BOOST_PROFESSOR`	`1.3`	BM25 score factor of professor material (before the coverage gate)
`INGEST_MAX_OCR_PAGES`	`20`	scanned pages read per file
`PASSAGE_BUDGET_SHARE, BOARD_BUDGET_SHARE`	`0.25, 0.25`	share of the prompt budget for explain's `passage` / `board_text`
`RETRIEVAL_QUERY_CHARS`	`400`	a passage or board text is cut to this many characters as a search query
`PREP_LEARN_MINUTES, PREP_REVIEW_MINUTES, PREP_ASSESSMENT_MAX`	`20, 8, 10`	prep plan: planned minutes per page, assessment size
`MAX_TOKENS_COMPACT, MAX_TOKENS_MIDSIZE`	`300, 550`	output caps per class

Variable	Default	Meaning
OUTPUT_CAP_MODE	both	options (options.num_predict), max_tokens or both; FelCloud honours only num_predict
INPUT_TOKEN_BUDGET	2800	estimated input cap enforced before sending
ESTIMATE_CHARS_PER_TOKEN	2.3	token estimator (worst measured case, Arabizi)
CONTEXT_WINDOW, CONTEXT_FILL	4096, 0.85	estimated prompt + output cap must fit window × fill
INPUT_TRUNCATION_CHARS_PER_TOKEN	5	prompt_tokens below chars/this = input cut (input_truncated); 0 = off
BUDGET_SAFETY_MARGIN	0.1	prompts assembled within budget × (1 − margin)
Retry and routing		
MAX_ATTEMPTS	4	attempts per request
MAX_RETRY_WAIT_S	30	INTERACTIVE: longest stated quota delay it waits for, else degraded with an eta
MAX_RETRY_WAIT_S_BACKGROUND	65	BACKGROUND: the observed quota delay is 60 s
CONCURRENCY_RETRY_WAIT_S	2	cap on a concurrency 429's wait (it states 60 s)
SATURATION_RETRIES	1	mid-size retries before fallback
COMPACT_MAX_RETRIES	3	compact retries (no fallback exists)
QUOTA_LOW_THRESHOLD	2	local tokens below this → explain goes to fast mode
BREAKER_THRESHOLD, _WINDOW_S, _COOLDOWN_S	3, 60, 30	mid-size circuit breaker
Cache and prompts		
CACHE_PATH	data/cache.sqlite	SQLite file (:memory: in tests)
CACHE_TTL_DAYS	7	exact-cache lifetime
NEGATIVE_CACHE_TTL_S	600	negative markers; 0 = off
PROMPTS_DIR	prompts	versioned prompt files
Retrieval and content		
RETRIEVER	bm25	bm25 or embedding
DOCS_DIR, RAG_INDEX_PATH	data/docs, data/index/chunks.json	course material and its index
RAG_TOP_K	2	chunks per explain after the gate
RAG_MIN_COVERAGE	0.4	relevance gate
CHUNK_MAX_TOKENS	300	chunk size
GLOSSARY_PATH, GLOSSARY_WEIGHT	data/glossary.yaml, 0.5	learner terms → course terms (empty = off)
TERMS_AR_PATH	data/terms_ar.yaml	Arabic math terms for MSA answers
EXERCISES_PATH	data/exercises/derivees.yaml	exercise bank
TEMPLATES_PATH	data/templates.yaml	openers, practice lines, feedback
EXPLAIN_MAX_TOKENS	350	explain output cap
CLASSIFY_, HINT_, GRADE_, COMPRESS_, QUIZ_INTRO_, FRAMING_MAX_TOKENS	50, 80, 120, 30, 60, 40	per-task output caps (cut text is trimmed to the last sentence)
EXPLAIN_FREQUENCY_PENALTY	0.3	discourages loops; dropped automatically if rejected
Metrics and simulator		
METRICS_PATH	metrics/calls.jsonl	JSONL metrics
SIMULATE_CONSTRAINTS	false	wrap the client in the simulator
SIM_LATENCY_COMPACT_S, _MIDSIZE_S	2,6 / 8,30	added latency range per class

Variable	Default	Meaning
SIM_RATE_PER_MIN, SIM_BURST	20, 20	the simulated provider's real quota
SIM_SATURATION_PROB, SIM_COMPACT_...	0.15, 0.0	saturation probability per class
SIM_SATURATION_DIALECT	annex	annex (503) or litellm (429 cooldown)
SIM_MAX_TOKENS_CEILING, SIM_PROMPT_TOKENS_CEILING	2048, 2048	400 above these
SIM_MIDSIZE_SLOTS, SIM_TRANSIENT_ERROR_PROB, SIM_SEED	1, 0.0, -	mid-size capacity, random 5xx, seed

Simulator profiles simulator/profiles.py names five profiles used by the load test and by /sim <name> in the console: normal (15 % mid-size saturation), calm (no saturation, no added latency), saturated (90 % / 20 %), quota_low (provider 10/min, burst 4) and capacity. The simulator is an httpx transport that wraps the real one, so the gatekeeper parses genuine responses on the same code path as FelCloud. Local Ollama never refuses anything: without the simulator, the resilience code is never exercised.

16 Customizing the engine

16.1 Teach a new subject (no code)

1. **Course material**: add data/docs/<name>.<lang>.md with the front matter and ## sections tagged <!-- concept: id -->; run python -m scripts.index_docs.
2. **Glossary**: add entries - fr: [course term] + variants: [...] in data/glossary.yaml (a variant may belong to one entry only; loading rejects duplicates).
3. **Exercises**: add to the bank YAML (id, concept, level, template_key + expression or statement: {fr, en, ar}, solution, hints, common_mistakes). A test checks that every chunk's concept has at least one exercise.
4. **Templates**: openers and lines per language in data/templates.yaml.
5. Re-run pytest -q and python -m scripts.eval_retrieval.

```
---
topic: derivative
level: beginner
language: FRENCH
title: Formules de dérivation
---
## Produit et quotient
<!-- concept: produit -->
Pour un produit de deux fonctions u et v : (u·v)' = u'·v + u·v'. Ce n'est pas u'·v'. ...
```

16.2 Change a prompt

Prompts live in prompts/<task>.md: a front matter with task and version, then ## name sections (system, user, lang.FRENCH, example.ARABIZI, fallback.DARIJA...) with $placeholders (Python string.Template). **Bump the version** when you change the meaning: it is part of the

cache key, so old answers are invalidated without a manual flush. Re-run the relevant eval script afterwards: small models are sensitive even to the order of few-shot examples.

16.3 Add your own task

A task is a `TaskSpec` plus a row in `ROUTES` plus a prompt file. `summary` already has its routing row (compact) but no implementation, so here is a complete one. This exact code was run against a fake provider while writing this guide: one compact request, the repeat came from the cache.

```
# prompts/summary.md
# ---
# task: summary
# version: 1
# ---
# ## system
# Summarize the text for a $level student in at most 3 short sentences. Reply in
    $language.
# ## user
# $text

from engine import Engine
from engine.tasks import TaskSpec, default_registry

def summary_build(p, inputs, ctx, limit=0):
    return [{"role": "system", "content": p.render("system", level=ctx.level or "high-
    school",
                                                   language=(ctx.language or "FRENCH"
    ).title())},
            {"role": "user", "content": p.render("user", text=str(inputs["text"]).
    strip())}]

SUMMARY = TaskSpec(name="summary", prompt="summary", max_tokens=120, temperature=0.2,
                   required_inputs=("text",), build=summary_build)

registry = default_registry()
registry.register(SUMMARY)        # "summary" already has a row in router.policy.
    ROUTES (compact)
engine = Engine.create(registry=registry)
# r = await engine.run("summary", {"text": "..."}, {"language": "FRENCH"})
```

`TaskSpec` hooks you can add, all optional: `output_schema` (pydantic model → JSON mode, validation, repair), `safe_default`, `shortcut` (answer without the LLM), `finalize_text` / `finalize_data` (guards, returning a reason), `fallback_text`, `prepare` (adjust context), `grounded` + `render_grounded` (RAG), `decorate` (deterministic scaffolding), `check_grounding` (formula check), `local` (no model at all), `rules_on_hit`. A task registered without a `ROUTES` row is refused at start-up.

16.4 Other common changes

You want to…	Change
switch provider or models	`.env` only (`LLM_BASE_URL`, `MODEL_*`); list several models per class for failover order
send explain to compact always	`ROUTES["explain"] = Route(C)`, in `router/policy.py` only
add a model class (e.g. code)	`ModelClass` in `config.py` + `MODEL_<CLASS>` + routes using it
adapt to a new error shape	`classify_response` / header constants in `gatekeeper/errors.py`
store metrics elsewhere	pass `sink=` any object with `write(CallRecord)`
tune fallback eagerness	`QUOTA_LOW_THRESHOLD`, `SATURATION_RETRIES`, `BREAKER_*`
make explain shorter/longer	`EXPLAIN_MAX_TOKENS` + the "$\leq$ 120 words" line in `prompts/explain.md`
expose it over HTTP	wrap `engine.run` in your web framework, one `Engine` per process (see below)

One engine per process, one process per key The rate limiter, queue, breaker and in-flight coalescing live in memory. Two worker processes sharing one API key would each believe they have 20 requests/min. Run a single engine process per key (behind your web server), or move the limiter state to a shared store first (§17).

17 Known limitations and improvement ideas

Limitation	Idea
in-memory limiter/queue/breaker	a Redis-backed sliding window and breaker for multi-process deployments
no semantic cache (no embedding model on FelCloud)	add it for explain once an embedding model exists; keep the key on language + level + style; threshold 0.94
quota-exhausted shape unconfirmed	confirm in a FelCloud slot; one-line change in `errors.py`
local latency $\neq$ FelCloud	rerun `load_test` and capacity in a FelCloud slot
Darija generation is weak	body stays French; templates need a native Tunisian review
lexical retrieval misses (e.g. "x(x+1)" never says "produit")	glossary entries from real learner questions; hybrid BM25 + embeddings
one topic in the bank, French statements for hints and solutions	more banks; translate hints and feedback
azi-1 (Arabizi "I don't understand") still labelled DIAGNOSTIC	handled by the confusion override; a larger eval set before tuning
saturated-profile priority inversion unexplained	rerun with larger n
SQLite calls are synchronous inside async code	fine at this scale; move to `aiosqlite` if the cache grows

A File map

Path	Contents
`config.py`	`Settings`, `ModelClass`, `load_settings` (all env variables)
`engine/core.py`	`Engine`: `create`, `run`, cache/negative/coalescing, fast mode, generation, outcomes
`engine/tasks.py`	`TaskSpec`, `Registry`, the six tasks, guards, loop detection
`engine/result.py`	`Result`, `Event`
`engine/grounding.py`, `mathcheck.py`	formula extraction and checks; safe sympy parsing
`engine/exercises.py`, `language.py`, `prompts.py`	bank, language rules, prompt loader
`engine/answers.py`, `claims.py`, `prep.py`, `course.py`, `vision.py`	check_answer logic, check_claim formulas, prep plan and page exercises, course graph, image preparation (v1.3)
`engine/fake.py`	`FakeEngine`: canned realistic results for the UIs, no LLM (v1.3)
`rag/ingest.py`	PDF / DOCX / Markdown → chunks, concept tags, OCR of scanned pages (v1.3)
`docs/ENGINE_API.md`	the integration contract, generated from real outputs (v1.3)
`data/courses/`	course graphs (v1.3)
`router/policy.py`, `breaker.py`	routing table, decisions, follow-up; circuit breaker
`gatekeeper/`	client, errors, limiter, queue, retry, cache
`rag/`	normalize, chunker, store, glossary, retriever (BM25, embedding), budget
`metrics/`	`CallRecord`, JSONL and memory sinks
`simulator/`	constraint transport, profiles
`prompts/`	classify, hint, explain, grade, compress, quiz, ocr_transcribe, check_claim, check_answer (versioned)
`data/`	docs, index, glossary, exercises, templates, Arabic terms
`scripts/`	smoke, chat, load_test, metrics_report, eval_*, index_docs, recon
`tests/`	768 unit tests + 5 integration tests (`-m integration`)
`DECISIONS.md`	every non-obvious design choice, with evidence